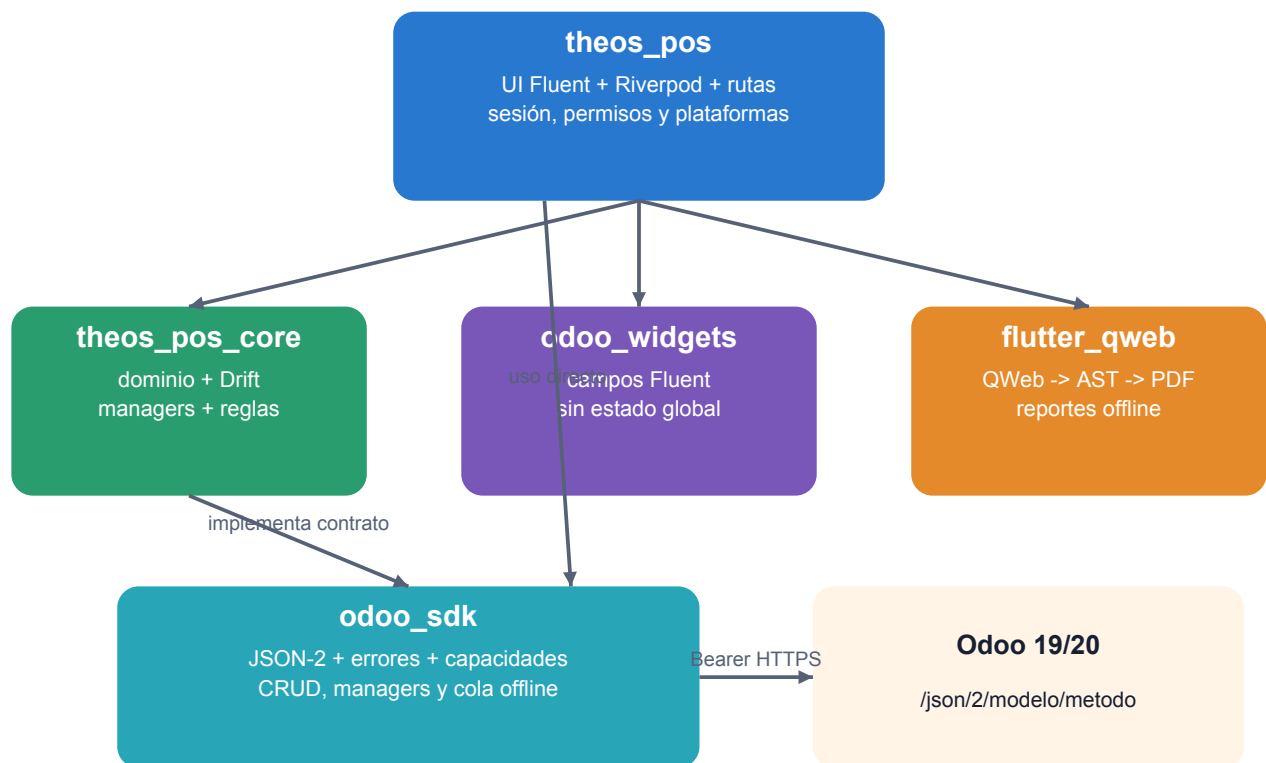


Arquitectura de Theos App

Cómo cooperan los cinco paquetes y donde construir nuevas funciones

Resultado central: theos_pos compone la aplicación; theos_pos_core contiene el dominio y Drift; odoo_sdk encapsula JSON-2 y el comportamiento offline; odoo_widgets aporta controles Fluent reutilizables; flutter_qweb convierte plantillas QWeb y datos en PDF.



Alcance del analisis

Documento levantado desde los pubspec, APIs publicas, providers, managers, repositorios, esquema Drift, sincronización y puntos reales de generación de reportes. Describe el runtime actual, no una arquitectura idealizada.

Versión de arquitectura observada: 3.0 - Bearer JSON-2, SessionScope y polling HTTP.

1. Mapa mental: una aplicación, cinco responsabilidades

Paquete	Rol	Qué contiene	Relacion
theos_pos	Aplicación Flutter	Pantallas Fluent, Riverpod, GoRouter, sesión, permisos, adaptadores de plataforma y composición.	Depende de los otros cuatro.
theos_pos_core	Dominio y datos	Modelos Freezed, tablas Drift, managers concretos, calculos de precios/impuestos/totales y servicios puros.	Depende de odoo_sdk; no depende de Flutter.
odoo_sdk	Infraestructura Odoo	Cliente JSON-2, transporte, capacidades por versión, errores, CRUD, base OdooModelManager, cola y sync genérico.	Paquete Dart reutilizable.
odoo_widgets	Biblioteca visual	Campos y componentes Fluent reactivos, responsivos y agnosticos a Riverpod.	No contiene reglas de negocio.
flutter_qweb	Motor documental	Parser XML QWeb, evaluador, AST, render PDF, formatos, fuentes y manejo de archivos.	No consulta Odoo por si solo en esta app.

Dependencias reales

- theos_pos -> todos: es la raíz de composición y decide cuando crear cliente, abrir base, ligar managers, iniciar sync y construir UI.
- theos_pos_core -> odoo_sdk: sus managers implementan OdooModelManager y sus modelos usan anotaciones/mapeos del SDK.
- odoo_widgets y flutter_qweb son hojas independientes: se pueden reutilizar sin importar el dominio Theos.
- El SDK exporta WebSocket como capacidad generica, pero el runtime Theos actual usa HTTP JSON-2 + polling; no hay WebSocket activo en la aplicación.

2. theos_pos: el director de orquesta

Este paquete es el ejecutable. Contiene las pantallas que ve vendedor/cajero, pero su responsabilidad mas importante es componer dependencias con el usuario y servidor activos.

Zona	Responsabilidad	Ejemplos
lib/features	UI y casos de uso por dominio	sales, collection, clients, products, invoices, sync, reports
lib/core/session	SessionScope y ciclo de vida	login online/offline, restauracion, teardown
lib/core/managers	Puente Riverpod -> managers core	initializeModelManagers, providers tipados
lib/core/database	Publicacion de Drift por scope	appDatabaseProvider, DatabaseHelper
lib/shared	Shell y piezas transversales	menu, permisos, configuración, cola, conflictos

Regla de oro

Una pantalla no debe construir URLs de Odoo ni decidir impuestos, cupos, estados financieros o reintentos. Observa providers; estos delegan a repositorios/managers/servicios; Drift alimenta la UI.

Riverpod aqui no es la base de datos

Riverpod mantiene composición y estado de presentación. Drift es la fuente durable e inmediata. Los StreamProvider observan consultas Drift; cuando un sync hace upsert, la interfaz se actualiza por el stream local.

3. theos_pos_core: dominio portable y persistencia

Es Dart puro: no importa Flutter. Por eso las reglas pueden probarse sin widgets y reutilizarse en otra aplicación o proceso.

Pieza	Qué resuelve	Ejemplo de cambio
Modelos Freezed	Entidades Odoo tipadas, defaults, enums y mapeo	Agregar campo a sale.order o res.partner
Tablas Drift	Persistencia por SessionScope	Guardar un catálogo o una operación offline
Managers	CRUD local/remoto, streams y sincronización por modelo	ProductManager, SaleOrderManager
Servicios puros	Calculos y reglas deterministas	impuestos, redondeo, listas de precio, totales
Registro de campos	Una fuente de verdad Odoo <-> local	evitar duplicar nombres y conversiones

Managers singleton, recursos por sesión

Los objetos manager son singleton, pero nunca deben conservar recursos del usuario anterior.
initializeModelManagers liga cada uno al OdooClient, AppDatabase y OfflineQueueWrapper del scope activo.
resetModelManagersSession limpia cliente, base, cola y caches antes de cerrar Drift.

No crear un segundo DataContext paralelo en la app. El runtime usa binding explícito para que login online, login offline y restauracion pasen por la misma puerta.

4. odoo_sdk: la frontera con Odoo

El SDK oculta la forma concreta de HTTP y ofrece una API estable: searchRead, call, create, write, unlink, detección de versión, capacidades y errores normalizados.

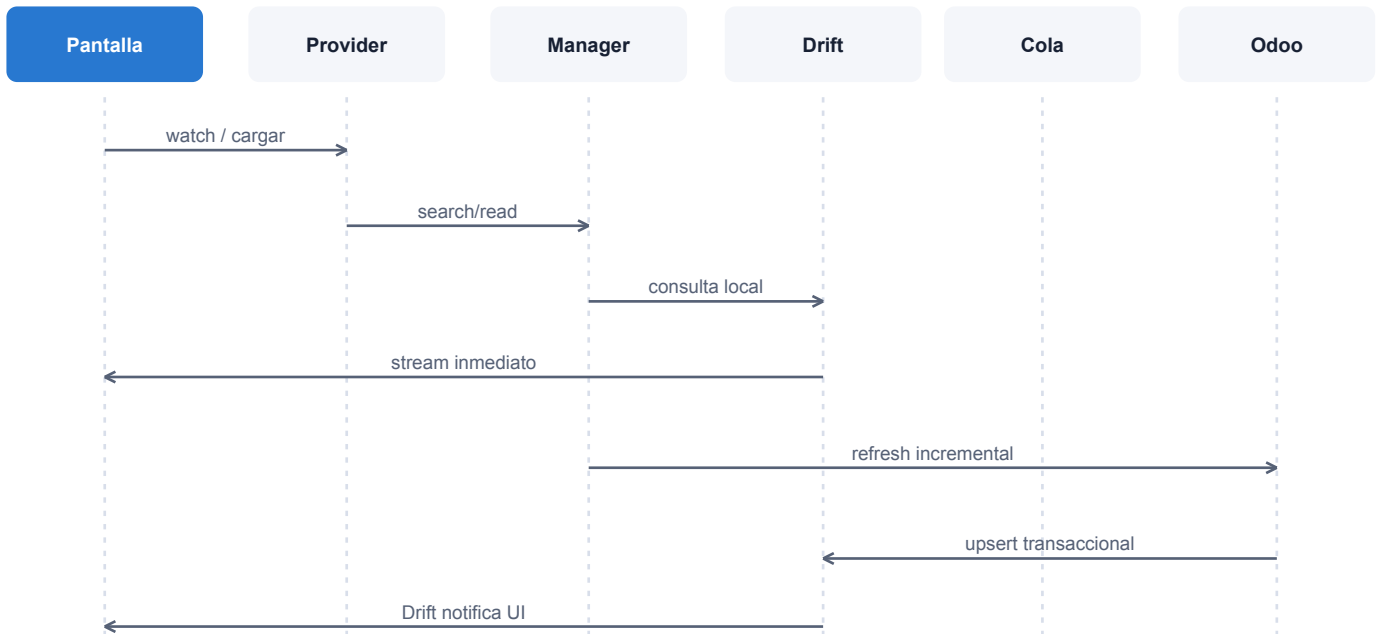
Capa SDK	Función	La app obtiene
OdooClient	Configura baseUrl, database y API key	único punto remoto publicado por provider
Json2Transport	POST /json/2//	sin URLs construidas en widgets
Capabilities	Diferencias Odoo 19/20	adaptadores en vez de ifs dispersos
OdooModelManager	local-first + sync + streams + conflictos	contrato reutilizado por managers core
OfflineQueue	comandos durables, retries y dependencias	continuidad cuando no hay servidor
Error mapper	clasifica autenticación, acceso, red y servidor	mensajes/control consistentes

Contrato efectivo

POST /json/2/modelo/método con Authorization: Bearer y X-Odoo-Database. Aunque una lectura también usa POST, el retry automático se limita a metodos clasificados como lectura; una mutacion desconocida no se reenvia a ciegas.

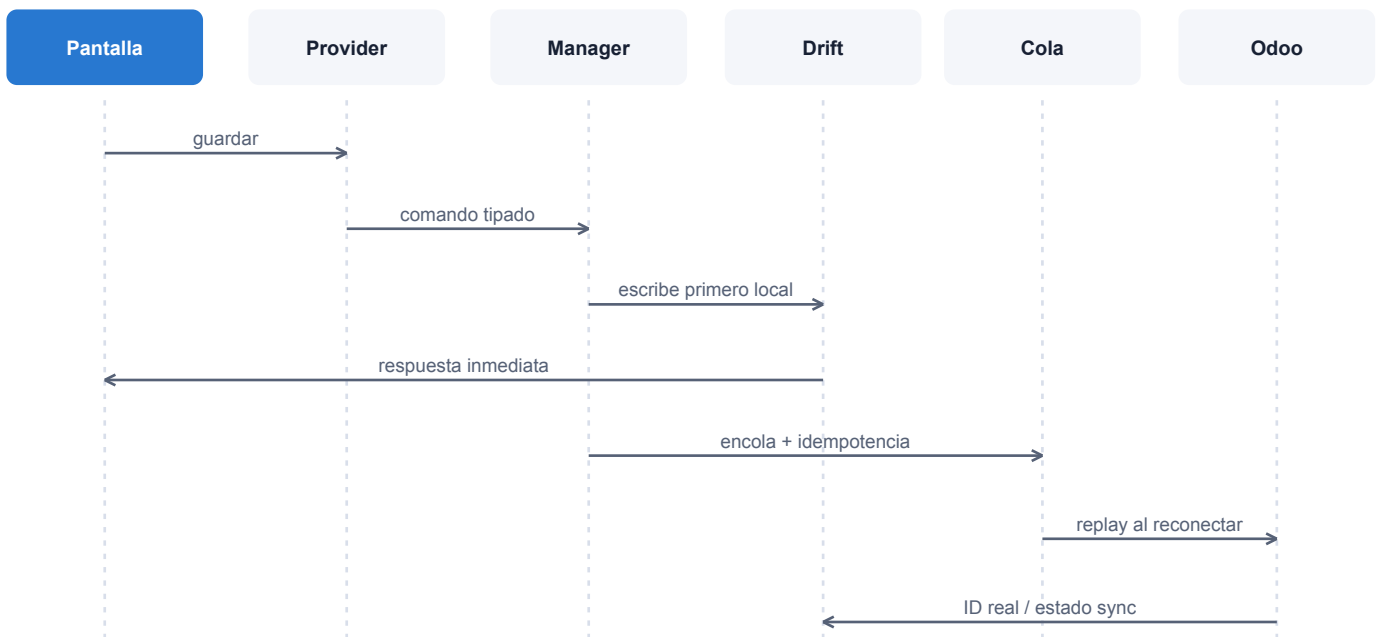
Odoo sigue siendo autoridad para ACL, record rules y validaciones finales. Offline-first significa poder registrar trabajo localmente y reconciliarlo después, no borrar los candados del servidor.

5. Lectura reactiva: la UI responde primero desde local



- La pantalla hace watch de un provider; no espera una llamada remota para poder dibujar.
- El provider consulta manager/repositorio; la respuesta inmediata sale de Drift.
- Si hay conexión, el manager u orquestador actualiza desde Odoo en segundo plano.
- El upsert transaccional dispara el stream Drift; Riverpod reconstruye solo los consumidores.
- Listas y contadores deben usar el mismo scope y la misma semantica de filtros.

6. Escritura offline: aceptar trabajo sin perder trazabilidad



- Una creación offline recibe UUID e ID local negativo; la UI puede seguir trabajando.
- La operación durable conserva tipo, payload mínimo, dependencias, intentos y próximo retry.
- Al recuperar red: primero se drena la cola, luego se sincronizan catálogos incrementales.
- Idempotencia evita duplicar ventas/cobros ante respuestas ambiguas; conflictos de `write_date` no se pisan silenciosamente.
- Una operación no demostrablemente idempotente debe ir a reconciliación, no repetirse automáticamente.

El orden importa: operación hija no se envía si su padre fallo. Borrar una fila local tampoco prueba que Odoo no ejecuto la operación.

7. odoo_widgets: controles visuales, no negocio

Biblioteca Fluent UI agnostica al gestor de estado. Aporta campos coherentes y componentes compuestos; recibe valores, configuración y callbacks desde theos_pos.

Componente	Uso	No debe hacer
OdooText/Number/Date/Boolean/Select/onField	edición tipada y consistente	llamar Odoo o decidir permisos
NumberInputBase	teclado, parseo y formato numerico	calcular impuestos o saldo
OdooMasterSelector	selección visual de maestros	ser propietario del catálogo
OdooSummaryRow/Card	resúmenes y totales presentados	calcular el total fuente
Responsive + spacing	tokens compartidos de layout	reemplazar la política adaptativa de la app

Cuándo ampliarlo

Solo cuando el control sea reutilizable en mas de una feature o aplicación Odoo y pueda expresarse sin Riverpod ni modelos Theos. Si el widget conoce `sale.order`, `SessionState` o reglas ecuatorianas, probablemente pertenece a `theos_pos`.

8. flutter_qweb: de plantilla Odoo a PDF offline

Paso	Responsable	Accion
1	theos_pos sync	Lee ir.ui.view, acciones y formatos desde Odoo mediante odoo_sdk.
2	Repositorio app	Guarda CachedTemplate y PaperFormat en tablas Drift de theos_pos_core.
3	ReportService	Carga plantillas, sanitiza, preprocesa records e inyecta empresa/usuario/locale.
4	QWebParser	Convierte XML y directivas t-if/t-foreach/t-field/t-call a AST.
5	Evaluator + renderer	Evalua expresiones y compone paginas PDF con fuentes y margenes.
6	theos_pos	Previsualiza, abre, comparte o imprime bytes PDF, incluso sin red.

flutter_qweb no conoce Riverpod, Drift ni la clave API. theos_pos adapta almacenamiento y datos; odoo_sdk trae la plantilla; theos_pos_core aporta las tablas; flutter_qweb solo interpreta y renderiza.

Riesgo a vigilar

QWeb de Odoo es amplio y dinamico. Cada directiva o expresion nueva debe implementarse y probarse en flutter_qweb; no conviene parchear una plantilla desde una pantalla. El cache de AST acelera renders repetidos.

9. Dónde implementar una función nueva

Pregunta	Destino
Es pantalla, navegacion, dialogo o estado de presentación?	theos_pos
Es una entidad, campo Odoo, tabla local o regla calculable sin Flutter?	theos_pos_core
Es transporte JSON-2, capacidad por versión, error, retry o primitiva offline generica?	odoo_sdk
Es un control Fluent reutilizable y sin conocimiento del negocio?	odoo_widgets
Es parsing QWeb, evaluación, formato de página o render PDF?	flutter_qweb
Necesita varias capas?	Crear una pieza pequena en cada capa; componer en theos_pos

Receta completa

- 1. Definir el contrato: modelo/método Odoo, campos, permisos, comportamiento online/offline e idempotencia.
- 2. Si hay datos nuevos, agregar modelo anotado y tabla/manager en theos_pos_core; regenerar fuentes.
- 3. Si el contrato remoto no existe, agregarlo al SDK como capacidad generica; no esconder una URL en la UI.
- 4. Registrar manager en initializeModelManagers y exponer provider/repository en theos_pos.
- 5. Implementar caso de uso y cola durable antes de conectar botones.
- 6. Construir UI con Fluent/odoo_widgets; aplicar RouteAccessPolicy y permisos default-deny.
- 7. Probar cálculo puro, persistencia, offline/replay/conflicto y finalmente widget/flujo real.

10. Ejemplo: agregar 'Visita de vendedor' offline

Capa	Cambio concreto
Odoo	Modelo/método publico, ACL/record rules y UUID/idempotency_key si se crea offline.
odoo_sdk	Nada si CRUD JSON-2 ya alcanza; agregar capacidad solo si hay contrato remoto genérico nuevo.
theos_pos_core	Visit model + Drift table + VisitManager; mapeo, write_date, sync status y servicios puros.
theos_pos	Provider/repository, registro del manager, ruta autorizada, lista/formulario y estado de envío.
odoo_widgets	Solo si aparece un selector reutilizable por varias features; de lo contrario usarlo desde app.
flutter_qweb	Solo si la visita necesita acta PDF offline: plantilla sincronizada + record map.

Flujo esperado

Vendedor crea visita sin red -> Drift guarda ID negativo/UUID -> UI muestra Pendiente -> cola conserva comando -> vuelve internet -> replay crea en Odoo -> manager reemplaza ID temporal por real -> Drift notifica -> UI muestra Sincronizado. Si Odoo rechaza permisos o datos, va a conflicto/dead-letter con evidencia; no se declara enviada.

No se necesita tocar los cinco paquetes en cada feature. La arquitectura busca que la mayoría de funciones nuevas vivan en theos_pos_core + theos_pos y reutilicen SDK/widgets/QWeb existentes.

11. Invariantes y trampas

Conservar	Evitar
Un OdooClient por SessionScope	clientes creados dentro de widgets
Base separada por servidor/base/UID	scope basado en API key o datos compartidos entre usuarios
Drift como lectura inmediata	pantallas bloqueadas esperando red
Cola durable e idempotencia	retry ciego de create/payment
Permisos en menu y rutas	ocultar boton como unica seguridad
Reglas en servicios/core	duplicar lógica entre panel, POS y dialogos
Tema Fluent y tokens	colores/tamanos locales que rompen accesibilidad
Plantillas QWeb en repositorio	XML o márgenes parcheados en pantallas

Pruebas proporcionales al cambio

Modelo/tabla: generación + esquema limpio. Sesión: online/offline/restore/logout/cambio de usuario. Sync: exclusión mutua, queue-first, recovery, idempotencia y conflictos. UI: provider mockeado, estados loading/error/offline y tamaños de iPad/Split View. QWeb: plantilla representativa y comparación visual del PDF.

12. Referencia rápida del repositorio

Tema	Fuente inspeccionada
Arquitectura runtime	theos_pos/ARCHITECTURE.md
Dependencias	*/pubspec.yaml de los cinco paquetes
Composición managers	theos_pos/lib/core/managers/manager_providers.dart
Contrato managers	odoo_sdk/lib/src/model/odoo_model_manager.dart
Cliente JSON-2	odoo_sdk/lib/src/api/odoo_client.dart y src/api/client/
Dominio y Drift	theos_pos_core/lib/theos_pos_core.dart y lib/src/database/
Widgets publicos	odoo_widgets/lib/odoo_widgets.dart
Motor QWeb	flutter_qweb/lib/flutter_qweb.dart y src/qweb_report_engine.dart
Templates offline	theos_pos/lib/features/reports y features/sync

Usa el cuaderno de infografías complementario para explicar la arquitectura al equipo y esta guía para decidir la ubicación de cada cambio.